

4 — Guide technique

Auteur : Ahmed Ben Arfa
Projet : Tunisia Weather Prediction
Date : juillet 2026

Conventions, outils et pratiques du projet.

1. Environnement

Python 3.13, environnement virtuel à la racine du dépôt.

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Linux / macOS
pip install -r requirements.txt
```

Outil	Rôle
pandas, pyarrow	Manipulation et format Parquet
DuckDB	Entrepôt analytique local, lecture native du Parquet
scikit-learn, XGBoost	Modélisation
matplotlib, seaborn	Visualisation
Streamlit	Application web
psycopg2, SQLAlchemy	Connexion Supabase
python-dotenv	Lecture des secrets

2. Où s'exécute quoi

Étape	Outil	Emplacement
ETL, nettoyage, schéma en étoile, EDA	DuckDB	Local
Entraînement	Python + Parquet	Local
Application déployée, journal des prédictions	Supabase	En ligne

DuckDB traite 1,58 M de lignes sans difficulté en local. Supabase ne reçoit que le produit fini et léger : dimension gouvernorat, fait journalier agrégé, journal des prédictions. Son palier gratuit plafonne à 500 Mo — **le fait horaire ne doit jamais y être chargé.**

3. Conventions de code

- **Langue** : commentaires et documentation en français, sans accents dans le code source pour éviter les problèmes d'encodage sous Windows. Noms de variables et de fonctions en anglais.
- **Chemins** : toujours relatifs à la racine du dépôt, construits avec `pathlib.Path`. Jamais de chemin absolu en dur.
- **Idempotence** : tout script doit pouvoir être relancé sans effet de bord. Les chargements

DuckDB utilisent `CREATE OR REPLACE`.

- **Reprise** : les traitements longs — extraction en tête — vérifient ce qui existe déjà avant de retravailler.
- **Notebooks** : générés par un script `_build_*.py` versionné, pour que le notebook soit reproductible et que les différences Git restent lisibles.

4. La règle la plus importante : une seule fonction de features

`build_features(df, horizon)` vit dans `05_machine_learning/features.py`. Elle est importée par le script d'entraînement **et** par l'application.

Cette logique n'est **jamais** dupliquée, ni recopiée, ni réécrite « en plus simple » côté application. Mêmes colonnes, mêmes noms, même ordre.

C'est la garantie qu'un modèle performant en validation le reste en production. Une divergence, même minime — une colonne dans un autre ordre, une fenêtre glissante calculée différemment — produit un modèle qui marche en test et se dégrade silencieusement une fois déployé.

5. Prévention de la fuite temporelle

La cible est `temperature_2m` décalée de H heures. Une variable n'est admissible que si sa valeur est **connue à l'instant t**.

Deux réflexes à appliquer systématiquement :

```
# CORRECT : on decale avant de calculer la fenetre
df["temp_moy_24h"] = df["temperature_2m"].shift(1).rolling(24).mean()

# FAUX : la fenetre inclut l'instant courant
df["temp_moy_24h"] = df["temperature_2m"].rolling(24).mean()
```

```
# CORRECT : decoupage chronologique
train = df[df["time"] < "2024-01-01"]

# FAUX : place des observations futures dans l'entrainement
train, test = train_test_split(df, test_size=0.2)
```

Signal d'alerte : un R^2 supérieur à 0,95 sur une prévision à 24 h. Ce n'est presque jamais un bon modèle, c'est presque toujours une fuite.

6. Secrets

La chaîne de connexion Supabase est lue depuis `.env`, jamais inscrite en dur.

```
SUPABASE_HOST=...
SUPABASE_PORT=5432
SUPABASE_DB=postgres
SUPABASE_USER=...
SUPABASE_PASSWORD=...
```

`.env` figure dans `.gitignore`. `.env.example` documente les variables attendues sans les valeurs. Sur Streamlit Cloud, ces valeurs passent par les `*secrets*` de la plateforme.

7. Ce qui est versionné, et ce qui ne l'est pas

Élément	Versionné	Raison
data/*.parquet	Oui	42 Mo, données publiques, rend le dépôt autonome
data/*.csv	Non	~308 Mo, redondants avec le Parquet
*.duckdb	Non	Se régénère depuis le Parquet
05_machine_learning/models/	Non	Se régénère par entraînement
06_web_app/models/	Oui	Streamlit Cloud déploie depuis le dépôt
.env	Non	Secrets

La distinction sur les modèles est délibérée : le modèle servi par l'application doit être présent dans le dépôt, sans quoi le déploiement n'a rien à charger.

8. Workflow Git

- Branche principale : `main`.
- Commits réguliers et descriptifs, en français.
- Un commit par unité de travail cohérente, pas un commit fourre-tout en fin de phase.
- Les fichiers volumineux passent par `.gitignore`, jamais par Git LFS — le Parquet compressé suffit.

9. Reproductibilité

Un tiers doit pouvoir cloner le dépôt et refaire tourner l'analyse sans demander quoi que ce soit :

```
git clone https://github.com/AhmedBenArfa/Tunisia-Weather-Prediction.git
cd Tunisia-Weather-Prediction
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
```

Les données étant dans le dépôt, aucune extraction n'est nécessaire. Chaque dossier numéroté porte un `README.md` décrivant son contenu et ses commandes.